



Веб-платформа

Браузеры, JavaScript и Node.js

Frontend-разработчик

Что разберём сегодня

- Как браузер превращает файлы в работающую страницу
- Почему JavaScript появился именно в браузере
- Чем отличаются язык, движок и среда выполнения
- Зачем фронтенд-разработчику Node.js и сборка

После занятия у каждого названия будет своё место

Вы открыли сайт. Что происходит дальше?

- Сервер присылает готовую картинку?
- Кто располагает текст и рисует кнопку?
- Где выполняется код после нажатия?

Вопрос аудитории: что делает сервер, а что браузер?

Браузер как платформа

- Получает документы и другие ресурсы по сети
- Строит и отображает интерфейс
- Выполняет JavaScript и предоставляет Web API
- Управляет вкладками, хранилищем и доступом к устройствам

Веб-приложение использует возможности браузера

HTML, CSS и JavaScript

HTML

Смысл и структура

Заголовок, поле ввода, кнопка, список задач

CSS

Внешний вид

Размеры, расположение, цвета, адаптация к экрану

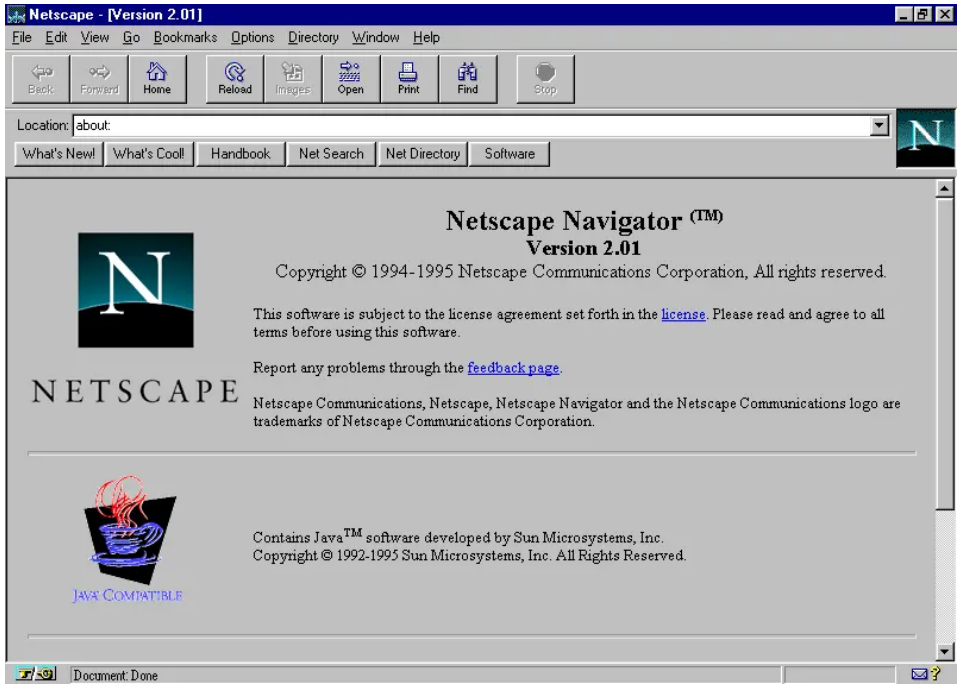
JavaScript

Поведение

Добавление задачи, обработка клика, запрос к API

Браузер объединяет эти части в один интерфейс

Середина 1990-х: вебу нужна интерактивность



- Netscape Navigator становится одной из площадок развития веба
- Для форм и реакций на действия нужен код внутри страницы
- Отправлять новую страницу после каждого действия неудобно

Задача: дать автору страницы простой язык сценариев

Брендан Эйх и десять дней

10 дней

Май 1995 года, Netscape

Первый прототип языка и движка JavaScript.
Дальше были месяцы интеграции и годы развития.

«Написал за неделю» относится к прототипу

Откуда название JavaScript

- Ранние названия: Mocha, затем LiveScript
- Название JavaScript закрепилось в 1995 году
- Netscape сотрудничала с Sun, а Java привлекала внимание
- У языков разные модели объектов и среды выполнения

JavaScript и Java развивались как самостоятельные языки

ECMAScript и общий стандарт

- Разным браузерам нужны одинаковые правила языка
- В 1997 году вышла первая редакция ECMA-262
- ECMAScript описывает синтаксис и поведение языка
- Комитет TC39 развивает стандарт вместе с участниками индустрии

ECMA-262 задаёт правила языка ECMAScript

Несколько поворотных точек

Год	Что изменилось
1995	Прототип JavaScript в Netscape
1997	Первый стандарт ECMAScript
2008	Chrome и движок V8
2009	Node.js: развитие JS вне браузера
2015	ES2015 / ES6: крупное обновление языка

Дальше язык развивается регулярными редакциями

Какой это язык

- Динамическая типизация: тип переменной не фиксирован
- Функции можно передавать как значения
- Объекты и прототипы входят в модель языка
- Память освобождает сборщик мусора

Синтаксис разберём отдельно. Сейчас важна модель исполнения

Браузер и два разных вида движков

Браузер на компьютере	Веб-движок	JS-движок
Chrome, Edge	Blink	V8
Firefox	Gecko	SpiderMonkey
Safari	WebKit	JavaScriptCore

Веб-движок работает со страницей, JS-движок исполняет код

Chrome и Chromium

- Chromium — открытый проект браузера
- Chrome — продукт Google на основе Chromium
- Edge и другие браузеры тоже используют Chromium
- Разные браузеры могут разделять большую часть движка

Другое название браузера не всегда означает другой движок

V8, SpiderMonkey и JavaScriptCore



- V8 разрабатывается в проекте Google, используется в Chrome и Node.js
- SpiderMonkey развивается Mozilla и используется в Firefox
- JavaScriptCore входит в WebKit и используется в Safari

Один язык может иметь несколько реализаций

Как движок выполняет JavaScript

01

Разбор

Движок проверяет синтаксис и строит внутреннее представление

02

Исполнение

Код может исполняться через байткод и интерпретатор

03

Оптимизация

JIT-компилятор превращает подходящий код в машинные инструкции

Упрощённая модель. Конкретные этапы зависят от движка

Сборка мусора: память тоже требует работы

- Программа создаёт объекты и хранит ссылки на них
- Движок определяет, какие объекты ещё достижимы
- Память недостижимых объектов можно освободить
- Если ненужный объект всё ещё доступен, он может остаться в памяти

Автоматическая память не исключает утечки

Язык, движок и среда выполнения

Уровень	За что отвечает	Пример
Язык	Правила программы	Функции, объекты, выражения
Движок	Выполнение JavaScript	V8, SpiderMonkey
Среда	Доступные программе возможности	Браузер, Node.js

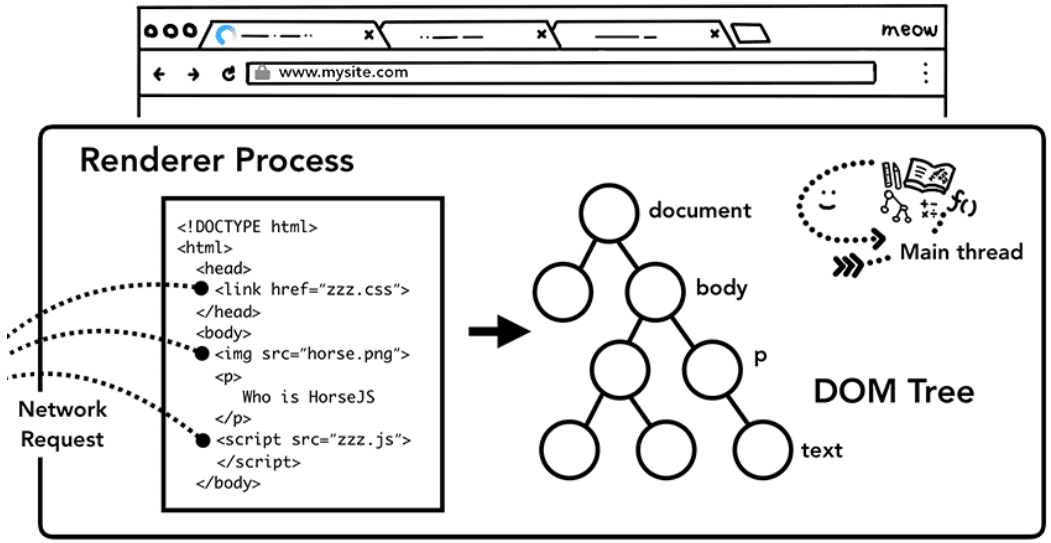
API среды определяют, что программа может делать вокруг себя

Какие возможности откуда берутся

Возможность	Кто её предоставляет
Массивы, функции, JSON	Сам язык и его стандартная библиотека
document, DOM, события кнопки	Браузер
setTimeout, console	Среда выполнения
Чтение файла через node:fs	Node.js

Одинаковый синтаксис не гарантирует одинаковый набор API

DOM: страница как дерево объектов

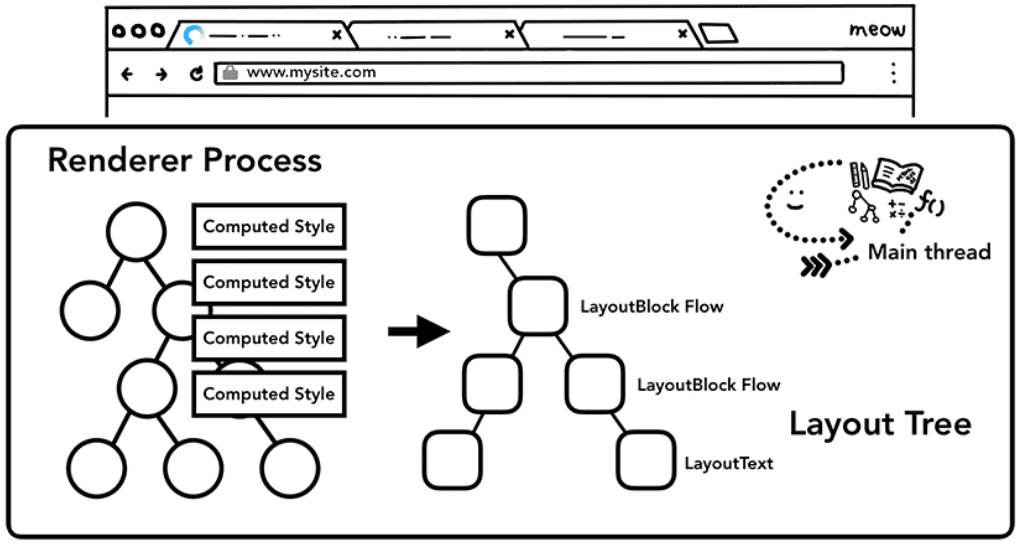


- Браузер разбирает HTML
- Создаёт дерево элементов
- JavaScript обращается к нему через DOM API

Иллюстрация: Mariko Kosaka / Chrome for Developers, CC BY 4.0

Изменение DOM может изменить то, что видит пользователь

Как появляется изображение страницы



- Стили задают оформление
- Layout определяет размеры и координаты
- Paint, растеризация и композиция дают кадр на экране

Иллюстрация: Mariko Kosaka / Chrome for Developers, CC BY 4.0

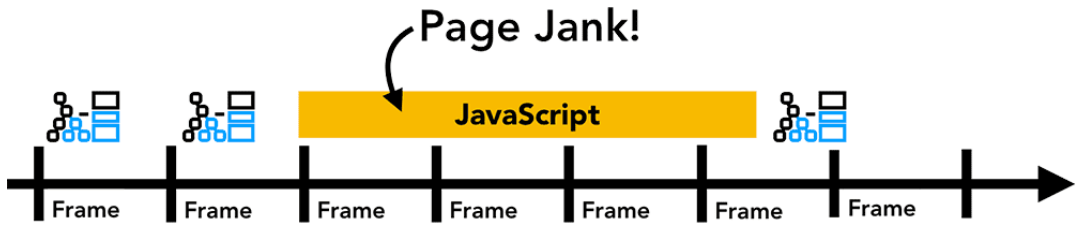
Браузер рассчитывает результат на устройстве пользователя

События и event loop

- Пользователь нажал кнопку: среда готовит обработчик к запуску
- Обработчик выполняется, когда приходит его очередь
- Сеть и таймеры позволяют продолжить работу позже
- Долгая синхронная работа задерживает другие задачи

Ожидание сети и вычисление на CPU ведут себя по-разному

Почему страница «зависла»



- Тяжёлый обработчик занял основной поток
- Новые реакции и обновление страницы задерживаются
- Быстрая сеть не ускорит этот расчёт

Иллюстрация: Mariko Kosaka / Chrome for Developers, CC BY 4.0

Делить работу на части или выносить расчёт в Web Worker

Почему сайт не читает весь ваш диск

- Код страницы приходит с удалённого сервера
- Браузер ограничивает его доступ к системе
- Доступ к камере и некоторым другим API требует разрешения
- Данные другого сайта защищают правила origin

Ограничения браузера нужны пользователю

Почему «у меня работает» недостаточно

- Пользователи открывают разные браузеры и версии
- Новые API появляются в них не одновременно
- Размер экрана и способы ввода тоже различаются
- Поддержку возможности проверяют по документации и тестами

Браузеры пользователей — часть условий задачи

DevTools: что происходит в браузере

Панель	На какой вопрос отвечает
Elements / Inspector	Как сейчас устроен DOM и какие стили применены?
Console	Какие ошибки возникли и что выведено программой?
Network	Какие ресурсы и данные загрузились?
Sources / Debugger	Какой код выполняется и где остановиться?

DevTools понадобится с самого первого проекта

Node.js: JavaScript вне браузера



- Среда выполнения JavaScript на основе V8
- Работа с файлами, процессами и сетью
- Запуск из терминала на компьютере или сервере

Скрипту больше не нужна открытая вкладка

2009: Райан Даль и Node.js

- Райан Даль начинает проект Node.js
- Готовый V8 можно встроить вне Chrome
- Событийная модель подходит для ожидания сетевого ввода-вывода
- Node.js получает API операционной системы и экосистему пакетов

JS вне браузера существовал раньше. Node.js сделал его массовым

Браузер и Node.js

Возможность	Браузер	Node.js
Выполнение JavaScript	Да	Да
DOM страницы	Да	Встроенного DOM нет
Произвольные файлы	Ограниченный доступ	В пределах прав процесса
Типичные задачи	Интерфейс пользователя	Инструменты, скрипты, серверы

API и права среды влияют на переносимость кода

Один небольшой скрипт

```
// hello.js
const name = "Frontend";
console.log("Привет, " + name);
```

Браузер
Вставить код в Console

Node.js
node hello.js

Логика одинаковая. Место запуска и вывод отличаются

Зачем фронтенд-разработчику Node.js?

- Код сайта выполняется в браузере
- У браузера уже есть JavaScript-движок
- Зачем тогда устанавливать ещё одну среду выполнения?

Подсказка: кто запускает инструменты разработки?

Node.js в работе фронтенд-разработчика

- Запускает локальный сервер разработки
- Помогает собрать проект для публикации
- Запускает линтеры, форматтеры и тесты
- Может выполнять серверную часть, если она есть

Node.js запускает инструменты, браузер выполняет код страницы

Node.js, npm и npm registry

Название	Роль
Node.js	Выполняет JavaScript вне браузера
npm CLI	Устанавливает пакеты и запускает scripts
npm registry	Хранит опубликованные пакеты и их метаданные
Пакет	Библиотека или инструмент с описанием и версией

Команда на компьютере и удалённый каталог — разные вещи

Что появится в папке проекта

Файл или каталог	Назначение
package.json	Описание проекта, зависимости и команды
package-lock.json	Зафиксированное дерево зависимостей для npm
node_modules/	Установленные пакеты
src/	Исходный код приложения
dist/	Результат сборки в типичном проекте Vite

Lock-файл обычно хранят в Git. node_modules обычно исключают

Разработка, сборка и работа сайта

01 **У разработчика**

Исходники, зависимости, Node.js и сервер разработки

02 **Во время сборки**

Инструменты подготавливают HTML, CSS, JavaScript и ресурсы

03 **У пользователя**

Браузер получает файлы и выполняет клиентскую часть

Для статически собранного сайта Node.js на устройстве пользователя не нужен

Где здесь TypeScript

- TypeScript добавляет систему статических типов поверх JavaScript
- Инструменты проверяют типы до запуска программы
- Для браузера TypeScript обычно преобразуют в JavaScript
- Ошибки сети и неправильные внешние данные всё равно возможны

Браузер исполняет результат подготовки проекта

Где здесь React, Vue и другие инструменты

- Библиотеки и фреймворки помогают строить интерфейс
- Они используют JavaScript и возможности браузера
- Сборщик готовит проект к запуску, движок выполняет код
- HTML, CSS и устройство браузера остаются важны

React, V8 и Vite решают разные задачи

Мини-задача: кто за это отвечает?

- Вычислить сумму двух чисел
- Найти кнопку на странице
- Исполнить JavaScript в Firefox
- Установить зависимости проекта
- Подготовить файлы сайта для публикации

Язык, браузер, движок, пакетный менеджер или сборщик?

Разбор мини-задачи

Действие	Ответ
Вычислить сумму	Операция языка, которую выполняет движок
Найти кнопку	DOM API браузера
Исполнить JS в Firefox	SpiderMonkey
Установить зависимости	Пакетный менеджер, например npm
Подготовить сайт	Инструмент сборки, например Vite

В реальном проекте эти уровни работают вместе

Что посмотрим в демонстрации

- Откроем простую страницу и её Network
- Изменим заголовок в Elements и перезагрузим страницу
- Выполним короткий скрипт в Console и через Node.js
- Посмотрим package.json и готовые файлы после сборки

Проследим путь от исходников до страницы в браузере

Практика: исследование страницы

- Найти в Network HTML, CSS и JavaScript
- Изменить текст в Elements и объяснить результат перезагрузки
- Вывести сообщение в Console
- Найти JS-движок своего браузера и подтвердить источником

Результат: четыре наблюдения и короткое объяснение

Что читать дальше

- MDN: JavaScript и устройство веб-платформы
- Chrome for Developers: Inside look at modern web browser
- V8, SpiderMonkey и WebKit: документация движков
- Node.js Learn: среда выполнения и отличия от браузера
- npm Docs и Vite Guide: инструменты проекта

Следующая тема: основы JavaScript с примерами